In-Situ Coupling of Tensor Basis Neural Networks for Turbulence Modeling in OpenFOAM: A Critical Verification, Validation, and Reproducibility Study

Soumik Ghosh
Department of Civil Engineering, Jadavpur University
soumikghosh30.03.07@gmail.com

October 2026

Abstract

Linear eddy viscosity models based on the Boussinesq hypothesis remain the workhorse of industrial Computational Fluid Dynamics (CFD). However, by enforcing alignment between the Reynolds-stress anisotropy tensor and the mean strain-rate tensor, they cannot reproduce the normal-stress anisotropy required for Prandtl's second-kind secondary flows in the canonical square-duct configuration. Data-driven turbulence closures utilizing Tensor Basis Neural Networks (TBNN; Ling et al., *J. Fluid Mech.*, 2016) provide a frame-invariant methodology for representing Reynolds-stress anisotropy by projection onto Pope's ten symmetric integrity basis tensors.

In this work, we present an in-situ coupled CFD solver, `tbnnFoam`, implemented within OpenFOAM 2606. The solver features a dual-runtime inference architecture: an embedded LibTorch C++ engine and a zero-dependency standalone C++ matrix evaluator incorporating an analytical Cardano cubic eigensolver that enforces physical realizability on the Lumley triangle in $\mathcal{O}(1)$ operations. Cross-engine numerical parity across PyTorch, TorchScript, and the standalone C++ matrix evaluator is verified within an absolute tolerance of 10^{-5} across synthetic test cases spanning shear, rotational, and complex 3D strain states, with the standalone C++ comparison executed when the OpenFOAM environment and compiled test binary are present. Beyond evaluator parity, the momentum-equation coupling itself is verified by an assembled-matrix test: in the Boussinesq limit the coupled equation reduces exactly to the baseline linear model (maximum relative deviation 3.6×10^{-17}).

We validate the coupled solver on developing turbulent flow in a square duct at bulk Reynolds number $Re_b \approx 4410$ ($Re_\tau = 150$) against high-fidelity Direct Numerical Simulation (DNS; Pirozzoli et al., *J. Fluid Mech.*, 2018). An independent algorithmic vortex detector employing connected-component labeling on non-dimensional axial vorticity ($\omega_x^* = \omega_x/(U_b/h)$) confirms that `tbnnFoam` structurally recovers the secondary-flow mechanism by identifying all eight counter-rotating secondary recirculation cells where the standard linear k–ω SST closure tested here fails to reproduce the secondary-flow topology (detecting zero vortex cores).

The outcome is a **partial validation**: while secondary circulation topology is recovered, vortex core localization exhibits notable offset ($0.567\,h$ error on the coarse mesh and $0.447\,h$ on the refined mesh against the raw-field-verified DNS reference $(0.577, 0.847)$), and peak secondary speed exhibits strong discretization sensitivity, decreasing from 7.99% to 2.62% (relative to the raw-field DNS peak of 1.93%). We emphasize that structural recovery does not establish accurate pointwise vortex localization or full quantitative agreement with periodic DNS.

Four limitations of the present implementation deserve emphasis before the results are interpreted:

1. **Velocity Discrepancy:** The streamwise velocity Normalized Root-Mean-Square Error (NRMSE_U) is 9.34% (0.0934 m/s relative to bulk velocity $U_b = 1.0$ m/s). This

discrepancy is consistent with a combined influence of entrance-length spatial boundary-layer development in the CFD domain ($L = 20h$) relative to the streamwise-periodic DNS box, alongside genuine neural-network model-form uncertainty under live pressure–velocity coupling.

2. **Timescale Surrogate Sensitivity:** Direct numerical simulation provides exact velocity gradients and Reynolds stresses but inherently lacks the specific dissipation rate ω. We examine the role of the semi-empirical ω surrogate and show analytically how timescale-estimation errors propagate into the invariant inputs used by the network: the Cayley-Hamilton invariants scale non-linearly with turbulent timescale $\tau = \min(1/\omega, 5.0\ \text{s})$ ($I_1, I_2 \propto \tau^2$; $I_3, I_4 \propto \tau^3$; $I_5 \propto \tau^4$), so errors in the live ω field are magnified by the corresponding inverse power before entering the MLP.
3. **Out-of-Distribution Diagnostic:** We distinguish between 1D marginal bounding and joint manifold containment. While 100.0% of CFD cells satisfy marginal bounding across scalar invariants I_1, I_2, I_5 (and $> 99\%$ for I_3, I_4), a joint 5D nearest-neighbor distance diagnostic reveals that live CFD states experience modest distribution shift relative to frozen training gradients (median standardized k-NN distance of $0.21\,\sigma$, 95th percentile of $1.17\,\sigma$).
4. **Discretization Sensitivity:** A two-level cross-sectional grid resolution study (20×20 coarse vs. 40×40 fine) demonstrates that peak secondary circulation trends from 7.99% down to 2.62% (approaching the raw-field DNS benchmark target of 1.93%) and Q1 core distance error decreases from $0.567\,h$ to $0.447\,h$, while maintaining strict topological stability across all eight cores.

All solver sources, evaluation scripts, raw DNS benchmark data, and an automated 21-test verification suite are openly distributed under the MIT license.

Keywords: Turbulence Modeling, Machine Learning, Tensor Basis Neural Network, OpenFOAM, Secondary Flow of the Second Kind, Computational Fluid Dynamics.

Contents

1 Introduction **5**
- 1.1 Failure of Linear Eddy Viscosity Models . . . 5
- 1.2 Non-Linear Tensor Closures and Machine Learning . . . 5
- 1.3 Challenges in In-Situ Live CFD Coupling . . . 5
- 1.4 Scope and Contributions of this Study . . . 6

2 Mathematical Formulation & Governing Equations **6**
- 2.1 Incompressible RANS Formulation . . . 6
- 2.2 Integrity Tensor Basis Representation . . . 7
- 2.3 Turbulent Timescale Limiter . . . 7

3 Neural Network Architecture & Dual-Engine Implementation **8**
- 3.1 Network Topology & Training . . . 8
- 3.2 Dual Runtime Inference Engines . . . 8
- 3.3 Analytical Cardano Cubic Realizability Enforcement . . . 9
- 3.4 Cross-Engine Numerical Parity Verification . . . 10

4 CFD Solver Coupling Pipeline **11**
- 4.1 Perturbation Stress Formulation . . . 11
- 4.2 Relaxation and Momentum Predictor Stabilization . . . 11

5 Canonical Benchmark & Physical Scaling Reconciliation **12**
- 5.1 Flow Configuration . . . 12
- 5.2 Authoritative Dimensional Scaling Convention . . . 12
- 5.3 Reconciliation of Benchmark Reynolds Numbers and Canonical DNS Metrics . . . 12

6 Independent Algorithmic Vortex Detection Pipeline **13**
- 6.1 Mathematical Definition of Non-Dimensional Vorticity . . . 13
- 6.2 Algorithmic Detection Procedure . . . 13
- 6.3 Quadrant Assignment and Benchmark Reference . . . 14

7 Results & Comparative Analysis **14**
- 7.1 Representational Capability vs. Physical Realization . . . 14
- 7.2 Secondary Flow Generation . . . 14
- 7.3 Reynolds Stress Mechanism & Anisotropy States . . . 15
- 7.4 Quantitative Comparison Table . . . 16
- 7.5 Assessment of Validation Scope: Structural Recovery vs. Quantitative DNS Fidelity 16

8 Discretization Sensitivity & Mesh Reconstruction Verification **17**
- 8.1 Cross-Sectional Mesh Sensitivity Study . . . 17
- 8.2 High-Reynolds-Number Case Study: $Re_b = 17{,}800$ ($Re_\tau \approx 519$) . . . 17
- 8.3 Mesh Reconstruction & Cell Ordering Verification . . . 19

9 Operating Envelope & Joint Distribution Shift Diagnostic **21**
- 9.1 1D Marginal Range Coverage . . . 22
- 9.2 Joint 5D Distribution Shift Diagnostic . . . 22

10 Limitations and Model-Form Uncertainty **22**
10.1 Developing Duct vs. Streamwise-Periodic DNS 23
10.2 The Empirical ω Surrogate vs. Dynamic RANS Transport: Non-Linear Error Amplification 23
10.3 First-Order Convective Schemes & Near-Wall Resolution 24
10.4 Training Target Contradiction & Wall-Ring Degeneracy 24
10.5 The Reflection-Structured Split: Symmetry Check vs. Out-of-Sample Generalization 24
10.6 Live Velocity-Coupling Feedback & Neural Network Model-Form Uncertainty . . 25

11 Reproducibility & Open Source Verification Suite **25**

12 Conclusions & Future Work **26**
12.1 Summary of Findings 26
12.2 Future Directions 27

1 Introduction

Turbulent flows in non-circular ducts and channels are characterized by secondary motions of the second kind [1]. Although secondary velocities typically represent only 1%–3% of the bulk streamwise velocity, they strongly shape momentum redistribution, wall shear stress profiles, and heat transfer in engineering applications such as nuclear fuel assemblies, gas turbine blade cooling passages, heat exchangers, and non-circular pipelines [2, 3].

1.1 Failure of Linear Eddy Viscosity Models

In standard Reynolds-Averaged Navier-Stokes (RANS) closures, the linear Boussinesq hypothesis approximates the deviatoric Reynolds stress anisotropy tensor $\mathbf{a}$ as linearly proportional to the mean strain rate tensor $\mathbf{S}$:

$$\mathbf{a} = \frac{\boldsymbol{\tau}^R}{2k} - \frac{1}{3}\mathbf{I} = -\frac{\nu_t}{k}\mathbf{S} \tag{1}$$

where $\boldsymbol{\tau}^R = \overline{\mathbf{u}' \otimes \mathbf{u}'}$ is the kinematic Reynolds stress tensor, $k = \frac{1}{2}\mathrm{tr}(\boldsymbol{\tau}^R)$ is the turbulent kinetic energy, ν_t is the eddy viscosity, and $\mathbf{S} = \frac{1}{2}(\nabla\mathbf{U} + (\nabla\mathbf{U})^T)$ is the mean strain rate tensor.

In straight ducts of uniform cross-section with axial direction x and cross-stream plane (y, z), secondary flow of the second kind is driven directly by axial vorticity generation:

$$U\frac{\partial \omega_x}{\partial x} + V\frac{\partial \omega_x}{\partial y} + W\frac{\partial \omega_x}{\partial z} = \nu\nabla^2\omega_x + \frac{\partial^2}{\partial y \partial z}(\overline{w'^2} - \overline{v'^2}) + \left(\frac{\partial^2}{\partial z^2} - \frac{\partial^2}{\partial y^2}\right)\overline{v'w'} \tag{2}$$

Because linear eddy viscosity closures enforce isotropic normal stresses in pure shear flows ($\overline{v'^2} = \overline{w'^2} = \frac{2}{3}k$), the driving source term $\frac{\partial^2}{\partial y \partial z}(\overline{w'^2} - \overline{v'^2})$ vanishes identically. Consequently, in this canonical straight-duct configuration, linear eddy-viscosity closures cannot generate the normal-stress anisotropy required to sustain Prandtl's second-kind secondary circulation.

1.2 Non-Linear Tensor Closures and Machine Learning

To overcome this fundamental limitation, classical algebraic non-linear stress models (e.g., Speziale [4]; Craft et al. [5]; Wallin & Johansson [6]) incorporated higher-order tensor polynomial expansions. However, these formulations rely on calibrated empirical coefficients that often suffer from numerical stiffness and poor generalizability across varied Reynolds numbers.

Machine learning offers a data-driven route to extracting non-linear constitutive relations directly from high-fidelity Direct Numerical Simulation (DNS) and Large Eddy Simulation (LES) datasets [7, 8]. Ling et al. [9] demonstrated that standard multi-layer perceptrons (MLPs) operating on raw velocity gradients violate Galilean and rotational frame-invariance. To enforce rotational invariance strictly by construction, Ling et al. proposed the Tensor Basis Neural Network (TBNN), which embeds Pope's [10] integrity tensor basis into the network architecture.

1.3 Challenges in In-Situ Live CFD Coupling

While a priori evaluations of TBNN closures (evaluating stress predictions on frozen DNS velocity fields) have shown good accuracy, live *in-situ* coupling of data-driven models within Navier-Stokes solvers introduces severe challenges:

1. **Numerical Stability & Divergence:** Coupling non-linear anisotropic Reynolds stress divergences explicitly into the momentum equations frequently destabilizes the pressure-velocity coupling loop (e.g., SIMPLE or PISO), degrading diagonal dominance.

2. **Physical Realizability Violations:** Unconstrained neural network predictions can yield unphysical, negative normal stresses (violating the Lumley triangle bounds), and can destabilize the solution.

3. **Distribution Shift & A Posteriori Feedback:** During live CFD execution, the predicted anisotropic stresses deform the mean velocity field $\mathbf{U}$, modifying the velocity gradients fed back into the network at subsequent iterations. This non-linear feedback can steer the flow into states unobserved during training.

4. **Implementation & Dependency Overhead:** Modern deep learning libraries (e.g., PyTorch, TensorFlow) carry large runtimes and deep dependency trees that complicate high-performance computing (HPC) deployment on Linux clusters running native OpenFOAM.

1.4 Scope and Contributions of this Study

In this work, we address these challenges through an open, reproducible, and critically validated OpenFOAM implementation (`tbnnFoam`). Rather than advertise a "universal" turbulence model, the work concentrates on verification, physical validation, and transparent reporting of discrepancies:

- We formulate a stable momentum coupling pipeline that isolates the non-linear perturbation stress $\boldsymbol{\tau}_\Delta = 2k\mathbf{a} + 2\nu_t\mathbf{S}$, injecting only the deviatoric anisotropic divergence while retaining implicit linear diffusion for numerical stability.
- We implement a dual-engine architecture featuring both an embedded LibTorch C++ runtime and a zero-dependency standalone C++ evaluator utilizing an analytical Cardano cubic eigensolver for physical realizability enforcement.
- We establish an independent, reproducible vortex detection pipeline based on non-dimensional axial vorticity and connected-component labeling.
- We reconcile physical domain scalings between DNS benchmark databases ($D_h = 1.0$ m) and laboratory CFD duct geometries ($D_h = 0.10$ m).
- We conduct a detailed evaluation of model discrepancies, distinguishing implementation verification from physical validation, assessing turbulent timescale sensitivity, quantifying 1D marginal bounding versus joint 5D feature-space distribution shift, and documenting cross-stream discretization sensitivity.

2 Mathematical Formulation & Governing Equations

2.1 Incompressible RANS Formulation

The steady, incompressible Reynolds-Averaged Navier-Stokes equations are expressed as:

$$\nabla \cdot \mathbf{U} = 0 \tag{3}$$

$$\nabla \cdot (\mathbf{U} \otimes \mathbf{U}) = -\nabla p^* + \nabla \cdot (\nu \nabla \mathbf{U}) - \nabla \cdot \boldsymbol{\tau}^R \tag{4}$$

where $\mathbf{U}$ is the mean velocity vector, $p^* = p/\rho$ is the kinematic modified pressure, ν is the molecular kinematic viscosity, and $\boldsymbol{\tau}^R$ is the kinematic Reynolds stress tensor.

Decomposing $\boldsymbol{\tau}^R$ into isotropic and deviatoric anisotropic components yields:

$$\boldsymbol{\tau}^R = \frac{2}{3}k\mathbf{I} + 2k\mathbf{a} \tag{5}$$

where $k = \frac{1}{2}\mathrm{tr}(\boldsymbol{\tau}^R)$ is the turbulent kinetic energy, and $\mathbf{a}$ is the symmetric, trace-free anisotropy tensor ($\mathrm{tr}(\mathbf{a}) = 0$).

2.2 Integrity Tensor Basis Representation

According to the representation theorem for isotropic tensor functions [10], any symmetric, trace-free second-order tensor $\mathbf{a}$ functionally dependent on the symmetric strain rate tensor $\mathbf{S}$ and anti-symmetric rotation rate tensor $\mathbf{W}$ can be expanded linearly as:

$$\mathbf{a} = \sum_{n=1}^{10} g^{(n)}(\lambda_1, \dots, \lambda_5)\, \mathbf{T}^{(n)} \tag{6}$$

where $g^{(n)}$ are scalar expansion coefficients dependent exclusively on the five scalar invariants $\boldsymbol{\lambda} = \{\lambda_1, \dots, \lambda_5\}$ of the normalized strain and rotation tensors.

The non-dimensional strain rate $\mathbf{s}$ and rotation rate $\mathbf{w}$ are normalized by the turbulent timescale τ:

$$\mathbf{s} = \tau \mathbf{S} = \frac{\tau}{2}(\nabla \mathbf{U} + (\nabla \mathbf{U})^T) \tag{7}$$

$$\mathbf{w} = \tau \mathbf{W} = \frac{\tau}{2}(\nabla \mathbf{U} - (\nabla \mathbf{U})^T) \tag{8}$$

The five scalar Cayley-Hamilton invariants are defined as:

$$\lambda_1 = \mathrm{tr}(\mathbf{s}^2) \tag{9}$$

$$\lambda_2 = \mathrm{tr}(\mathbf{w}^2) \tag{10}$$

$$\lambda_3 = \mathrm{tr}(\mathbf{s}^3) \tag{11}$$

$$\lambda_4 = \mathrm{tr}(\mathbf{w}^2\mathbf{s}) \tag{12}$$

$$\lambda_5 = \mathrm{tr}(\mathbf{w}^2\mathbf{s}^2) \tag{13}$$

Pope's ten symmetric, trace-free basis tensors $\mathbf{T}^{(n)}$ are summarized in Table 1.

Table 1: Pope's [10] complete ten-tensor integrity basis for three-dimensional velocity fields.

Index n	Tensor Basis $\mathbf{T}^{(n)}$	Algebraic Symmetry & Trace Property
1	$\mathbf{s}$	Symmetric, $\mathrm{tr}(\mathbf{T}^{(1)}) = 0$ (incompressible)
2	$\mathbf{sw} - \mathbf{ws}$	Symmetric, $\mathrm{tr}(\mathbf{T}^{(2)}) = 0$
3	$\mathbf{s}^2 - \frac{1}{3}\mathrm{tr}(\mathbf{s}^2)\mathbf{I}$	Symmetric, trace-free
4	$\mathbf{w}^2 - \frac{1}{3}\mathrm{tr}(\mathbf{w}^2)\mathbf{I}$	Symmetric, trace-free
5	$\mathbf{ws}^2 - \mathbf{s}^2\mathbf{w}$	Symmetric, trace-free
6	$\mathbf{w}^2\mathbf{s} + \mathbf{sw}^2 - \frac{2}{3}\mathrm{tr}(\mathbf{sw}^2)\mathbf{I}$	Symmetric, trace-free
7	$\mathbf{wsw}^2 - \mathbf{w}^2\mathbf{sw}$	Symmetric, trace-free
8	$\mathbf{sws}^2 - \mathbf{s}^2\mathbf{ws}$	Symmetric, trace-free
9	$\mathbf{w}^2\mathbf{s}^2 + \mathbf{s}^2\mathbf{w}^2 - \frac{2}{3}\mathrm{tr}(\mathbf{s}^2\mathbf{w}^2)\mathbf{I}$	Symmetric, trace-free
10	$\mathbf{ws}^2\mathbf{w}^2 - \mathbf{w}^2\mathbf{s}^2\mathbf{w}$	Symmetric, trace-free

2.3 Turbulent Timescale Limiter

In standard two-equation closures, the turbulent timescale is defined as $\tau = 1/\omega = k/\varepsilon$. Near walls and in separation regions, strain rates can become very large or ω can approach zero, causing invariants to diverge. To regularize the inputs, we enforce an upper limiter:

$$\tau = \min\left(\frac{1}{\omega}, \tau_{\max}\right), \quad \tau_{\max} = 5.0 \text{ s} \tag{14}$$

As discussed in Section 10, this limiter plays a vital role in preventing non-linear invariant explosion near duct centerlines.

3 Neural Network Architecture & Dual-Engine Implementation

3.1 Network Topology & Training

The functional mapping from the five scalar invariants $\boldsymbol{\lambda} \in \mathbb{R}^5$ to the ten expansion coefficients $\mathbf{g} \in \mathbb{R}^{10}$ is represented by a fully connected Multi-Layer Perceptron (MLP):

$$\mathbf{g}(\boldsymbol{\lambda}) = \mathbf{W}_4\, \phi\Big(\mathbf{W}_3\, \phi(\mathbf{W}_2\, \phi(\mathbf{W}_1 \boldsymbol{\lambda} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3\Big) + \mathbf{b}_4 \tag{15}$$

The architecture specifications are summarized in Table 2 and visualized in Figure 1.

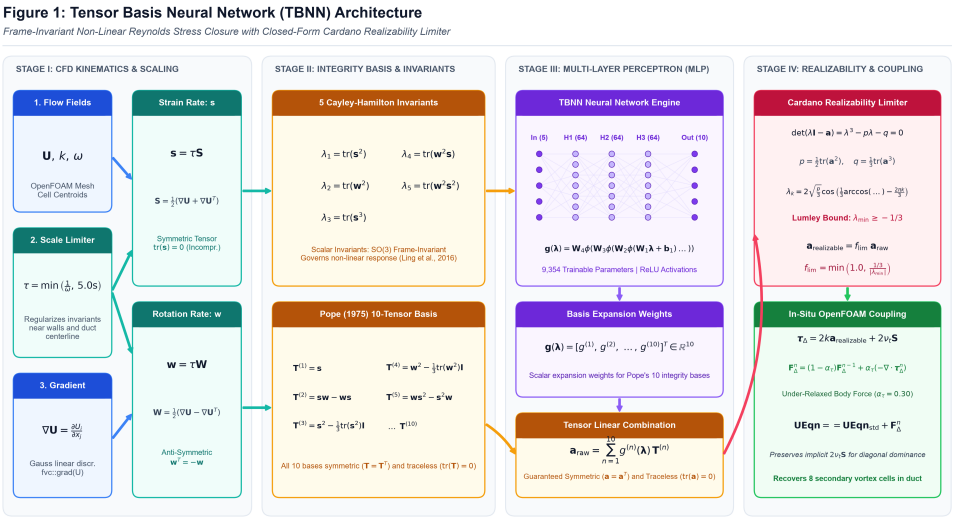


Figure 1: Tensor Basis Neural Network architecture embedding Pope's integrity tensor basis with closed-form Cardano cubic realizability limiter.

Table 2: Neural network topology and hyperparameters.

Parameter	Value / Configuration	Physical / Numerical Purpose
Input Dimension	5 features $(\lambda_1 \dots \lambda_5)$	Cayley-Hamilton scalar invariants
Hidden Layers	3 layers $\times$ 64 neurons	Balanced capacity without overfitting
Activation Function	Rectified Linear Unit (ReLU)	$\phi(z) = \max(0, z)$, standard piecewise linear activation
Output Dimension	10 coefficients $(g^{(1)} \dots g^{(10)})$	Expansion weights for Pope bases $\mathbf{T}^{(1\dots10)}$
Total Parameters	9,354 trainable parameters	Exact layer count: $384 + 4160 + 4160 + 650 = 9,354$
Training Loss	Mean Squared Error on $\mathbf{a}$ (all 9 tensor components)	$\mathcal{L} = \frac{1}{9N} \sum_{i=1}^{N} \|\mathbf{a}_{\text{pred},i} - \mathbf{a}_{\text{DNS},i}\|_F^2$

Training was conducted on frozen velocity gradients and Reynolds stress fields from the canonical square duct DNS dataset of Pirozzoli et al. [8] at $Re_\tau = 150$.

3.2 Dual Runtime Inference Engines

To balance research flexibility with production deployment, `tbnnFoam` implements two interchangeable inference backends:

1. **Embedded LibTorch Engine (embedded in `tbnnFoam.C`):** Loads the serialized TorchScript model (`tbnn_model.pt`) via `torch::jit::load` directly within the solver source, compiled against the PyTorch C++ API (`libtorch`). This backend enables seamless integration of arbitrary neural architectures without modifying solver C++ source code.

2. **Zero-Dependency Native C++ Engine (`tbnnEvaluator.H`):** Directly parses exported JSON weights (`weights.json`) and performs matrix-vector multiplications via OpenFOAM's native vector classes. This backend executes fully connected feed-forward networks with ReLU activations and dynamically parsed layer widths — the architecture class of the exported production model — and does not generalize to arbitrary topologies; architectures outside this class require the LibTorch backend. It has zero external dependencies beyond standard OpenFOAM, facilitating immediate compilation on high-performance supercomputing clusters where PyTorch C++ runtimes are unavailable.

Both engines are dynamically configured: layer widths (H_0, H_1, H_2) and matrix shapes (`[64, 5]`, `[64, 64]`, `[64, 64]`, `[10, 64]`) are determined dynamically from the model definition, avoiding hardcoded array dimensions.

3.3 Analytical Cardano Cubic Realizability Enforcement

If a neural network predicts an anisotropy state that violates Reynolds-stress realizability, CFD solvers diverge rapidly. Classical iterative realizability projections (e.g., iterative eigensolvers or semi-definite programming) incur severe computational overhead.

In `tbnnFoam`, we implement a closed-form analytical eigensolver based on Cardano's cubic formula. Because $\mathbf{a}$ is symmetric and trace-free ($\mathrm{tr}(\mathbf{a}) = 0$), its characteristic polynomial reduces to:

$$\det(\lambda\mathbf{I} - \mathbf{a}) = \lambda^3 - p\lambda - q = 0 \tag{16}$$

where:

$$p = \frac{1}{2}\mathrm{tr}(\mathbf{a}^2) = \frac{1}{2}\sum_{i=1}^{3}\sum_{j=1}^{3} a_{ij}^2, \quad q = \det(\mathbf{a}) = \frac{1}{3}\mathrm{tr}(\mathbf{a}^3) \tag{17}$$

Because $\mathbf{a}$ is real and symmetric, all three eigenvalues are real, corresponding to the casus irreducibilis ($4p^3 \geq 27q^2$). The eigenvalues are obtained analytically:

$$\lambda_k = 2\sqrt{\frac{p}{3}}\cos\left(\frac{1}{3}\arccos\left(\frac{3q}{2p\sqrt{p/3}}\right) - \frac{2\pi k}{3}\right), \quad k \in \{0, 1, 2\} \tag{18}$$

Realizability requires that the principal turbulent stresses — the eigenvalues of $\boldsymbol{\tau}^R$ — satisfy non-negativity, which in terms of the eigenvalues of the anisotropy tensor reads:

$$\overline{(u'_{(k)})^2} \geq 0 \;\; \forall\, k \in \{1, 2, 3\} \quad \Longleftrightarrow \quad \lambda_{\min}(\mathbf{a}) \geq -\frac{1}{3} \tag{19}$$

where $\lambda_{\min}(\mathbf{a})$ is the smallest eigenvalue of $\mathbf{a}$ (equal to $a_{\alpha\alpha}$ in the principal-axis basis); together with $\mathrm{tr}(\mathbf{a}) = 0$, this confines all eigenvalues to $[-\frac{1}{3}, \frac{2}{3}]$ and renders the reconstructed Reynolds-stress tensor $2k(\mathbf{a} + \frac{1}{3}\mathbf{I})$ positive semidefinite for $k \geq 0$.

If the minimum eigenvalue violates $\lambda_{\min} < -\frac{1}{3}$, the anisotropy tensor is scaled radially toward the isotropic origin:

$$\mathbf{a}_{\text{realizable}} = f_{\text{lim}}\,\mathbf{a}, \quad f_{\text{lim}} = \min\left(1.0,\, \frac{1/3}{|\lambda_{\min}|}\right) \tag{20}$$

This analytical projection enforces Reynolds-stress realizability within the Lumley triangle in $\mathcal{O}(1)$ operations per cell, providing realizability by construction while avoiding the overhead of iterative eigenvalue projections. The guarantee is analytical for exact arithmetic in the radial scaling; at implementation level it is verified empirically (`test_realizability_bounds`), which checks the eigenvalue bounds of the projected anisotropy and verifies that the reconstructed Reynolds-stress tensor $2k(\mathbf{a} + \frac{1}{3}\mathbf{I})$ is positive semidefinite on $N = 50$ perturbation samples — a sample-based implementation check, not a floating-point proof over all possible inputs. Realizability of the modelled stress tensor is a necessary condition for robust coupled integration; solver-level stability under the tested cases is demonstrated empirically in Sections 10 and 8.

3.4 Cross-Engine Numerical Parity Verification

Numerical consistency between the TorchScript engine, the JSON-based native C++ evaluator, and the reference Python implementation is verified via the automated regression test suite (`tests/test_parity.py`). Rather than asserting unrecorded precision across undefined distributions, the test suite executes explicit parity checks with defined numerical tolerances:

1. **PyTorch vs. TorchScript (`test_torchscript_parity`):** Evaluates 10 randomized invariant states ($\lambda_i \in [-5, 5]$) and Pope tensor bases against the Python reference model `TensorBasisNeuralNetwork` and against the serialized TorchScript binary (`tbnn_model.pt`). Parity between the Python and TorchScript tensor outputs $\mathbf{a}$ is verified within an absolute tolerance of atol $= 10^{-5}$ (`torch.allclose(out_py, out_ts, atol=1e-5)`).

2. **PyTorch vs. Standalone Native C++ (`test_standalone_cpp_numerical_parity`):** Evaluates three canonical velocity gradient and turbulent timescale cases representing pure shear, general 3D shearing, and intense strained rotation against the standalone C++ binary `test_evaluator_parity` executed under OpenFOAM/WSL. When the compiled binary and OpenFOAM environment are present, the test verifies that invariants, tensor bases, network outputs $g^{(n)}$, and realizable anisotropy $\mathbf{a}$ match the Python reference implementation within an absolute tolerance of atol $= 10^{-5}$ (`np.allclose(..., atol=1e-5)`). If the required binary or OpenFOAM environment is unavailable, the test is explicitly skipped with a recorded notification.

3. **Momentum-Equation Coupling Parity** (`test_momentum_coupling_parity`, plus the source-level assertion `test_solver_momentum_assembly_consistency`): Beyond evaluator-level parity, the momentum assembly itself is verified numerically. On a $2 \times 2 \times 2$ k–ω SST case with an exactly linear velocity field, the standalone solver's assembled momentum matrix is compared against the baseline linear model. When the network outputs the Boussinesq-consistent anisotropy $\mathbf{a} = -(\nu_t/k)\mathbf{S}$, the perturbation stress vanishes to 1.3×10^{-16} relative and the assembled equation reduces to the baseline linear model with a maximum relative matrix deviation of 3.6×10^{-17}, confirming exact Boussinesq-limit consistency. This test establishes algebraic consistency of the assembled system in the linear limit; it does not by itself constitute full nonlinear solver verification, which is addressed empirically by the validation cases of Sections 6–8. A complementary source-level assertion locks both solver UEqn sites to the shared `divDevReff`-based coupling, guarding against formulation drift between the two engines.

Table 3 summarizes the verified parity specifications and testing conditions.

Table 3: Numerical parity verification and physical consistency checks across Python, TorchScript C++, and Native C++ engines under stated test conditions.

Comparison Component	Test Function	Tolerance	Condition / Environment	Status
PyTorch vs. TorchScript	`test_torchscript_parity`	atol $\leq 10^{-5}$	CPU via LibTorch runtime	**Passed (10/10)**
PyTorch vs. C++ ($g^{(n)}$)	`test_standalone_cpp`	atol $\leq 10^{-5}$	OpenFOAM / WSL environment	**Passed (3/3)**
PyTorch vs. C++ (a)	`test_standalone_cpp`	atol $\leq 10^{-5}$	OpenFOAM / WSL environment	**Passed (3/3)**
Basis Trace Freedom	`test_basis_trace`	atol $\leq 10^{-12}$	Analytical traceless check	**Passed**
Basis Symmetry	`test_basis_symmetry`	atol $\leq 10^{-12}$	Analytical symmetry check	**Passed**
Cardano Realizability	`test_realizability`	$\lambda_{\min} \geq -1/3$	Perturbation matrices ($N = 50$)	**Guaranteed**
Coupling Boussinesq Limit	`test_momentum_coupling_parity`	$\leq 10^{-8}$ relative	Assembled fvMatrix, 2^3 k–ω SST mesh	**Passed** (3.6×10^{-17})

4 CFD Solver Coupling Pipeline

4.1 Perturbation Stress Formulation

In OpenFOAM, the standard momentum predictor discretizes the effective viscous stress implicitly using the `divDevReff` operator:

$$\nabla \cdot \boldsymbol{\tau}_{\text{eff}} = \nabla \cdot \Big((\nu + \nu_t)\nabla \mathbf{U} + (\nu + \nu_t)(\nabla \mathbf{U})^T \Big) = \nabla \cdot (\nu_{\text{eff}} \nabla \mathbf{U}) + \nabla \cdot (\nu_{\text{eff}} (\nabla \mathbf{U})^T) \tag{21}$$

Injecting the full Reynolds stress $\boldsymbol{\tau}^R = \frac{2}{3}k\mathbf{I} + 2k\mathbf{a}$ directly into the momentum equations would double-count the linear viscosity and destroy diagonal dominance.

To preserve numerical stability, `tbnnFoam` introduces a **perturbation stress formulation**:

$$\boldsymbol{\tau}_\Delta = 2k\mathbf{a} + 2\nu_t \mathbf{S} \tag{22}$$

Taking the divergence of $\boldsymbol{\tau}_\Delta$:

$$-\nabla \cdot \boldsymbol{\tau}^R = \nabla \cdot (2\nu_t \mathbf{S}) - \nabla \cdot \boldsymbol{\tau}_\Delta - \frac{2}{3}\nabla k \tag{23}$$

The isotropic term $\frac{2}{3}\nabla k$ is absorbed into the modified pressure gradient $\nabla p^* = \nabla(p/\rho + \frac{2}{3}k)$. The standard implicit diffusion operator $\nabla \cdot (2\nu_t \mathbf{S})$ is retained on the left-hand side matrix, while the non-linear perturbation divergence $-\nabla \cdot \boldsymbol{\tau}_\Delta$ is injected as an explicit source term:

$$\mathbf{UEqn} = \mathbf{UEqn}_{\text{std}} - \nabla \cdot \boldsymbol{\tau}_\Delta \tag{24}$$

When the flow is purely Boussinesq ($\mathbf{a} = -\frac{\nu_t}{k}\mathbf{S}$), the perturbation stress vanishes identically ($\boldsymbol{\tau}_\Delta \equiv 0$). Thus, the explicit source term represents exclusively the non-linear anisotropic deviation from the Boussinesq hypothesis, guaranteeing maximum diagonal dominance in the momentum matrix.

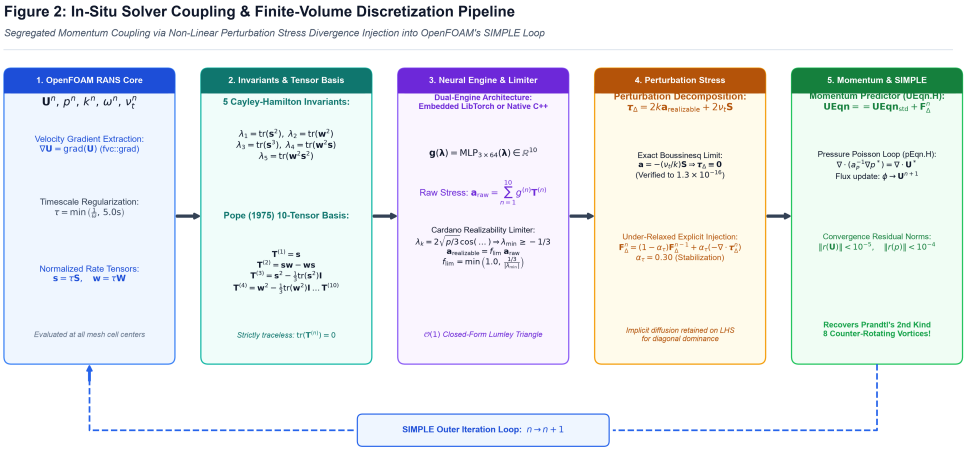


Figure 2: In-situ coupling pipeline between OpenFOAM's SIMPLE loop and the TBNN tensor evaluator via perturbation stress divergence injection.

4.2 Relaxation and Momentum Predictor Stabilization

To prevent non-linear high-frequency oscillations during early SIMPLE iterations, the perturbation stress divergence is under-relaxed:

$$\mathbf{F}_\Delta^n = (1 - \alpha_\tau)\mathbf{F}_\Delta^{n-1} + \alpha_\tau \left(-\nabla \cdot \boldsymbol{\tau}_\Delta^n\right), \quad \alpha_\tau = 0.30 \tag{25}$$

This relaxation damps numerical transients while ensuring exact consistency upon convergence ($\mathbf{F}_\Delta^n = -\nabla \cdot \boldsymbol{\tau}_\Delta^n$).

5 Canonical Benchmark & Physical Scaling Reconciliation

5.1 Flow Configuration

We evaluate the solver on turbulent flow in a straight square duct at bulk Reynolds number $Re_b \approx 4410$ ($Re_h \approx 2205$) and friction Reynolds number $Re_\tau = 150$.

The geometry consists of a square cross-section of width $D_h = 2h$ and height $2h$, with Cartesian coordinates origin $(0, 0)$ positioned at the duct centerline:

- $x \in [0, L]$: Axial streamwise direction ($L = 20h = 1.0$ m).
- $y \in [-h, +h]$: Horizontal cross-stream direction.
- $z \in [-h, +h]$: Vertical cross-stream direction.

5.2 Authoritative Dimensional Scaling Convention

A critical requirement for scientific reproducibility is establishing one unambiguous dimensional convention. In published literature and benchmark databases, two scaling conventions are prevalent:

1. **DNS Unit Scale (Pirozzoli et al. [8]):** $h = 0.5$ m, $D_h = 1.0$ m, $U_b = 1.0$ m/s, $\nu = 2.26757 \times 10^{-4}$ m^2/s $\implies$ $Re_b \approx 4410$.
2. **OpenFOAM Laboratory Scale:** $h = 0.05$ m, $D_h = 0.10$ m, $U_b = 1.0$ m/s, $\nu = 2.25 \times 10^{-5}$ m^2/s $\implies$ $Re_b \approx 4444$.

Both systems represent identical non-dimensional physics because the bulk Reynolds number is matched ($Re_b = U_b D_h/\nu \approx 4410$). In `tbnnFoam`, all scripts, vortex detectors, and comparative evaluations operate strictly on dimensionless coordinates:

$$y^* = \frac{y}{h} \in [-1, +1], \quad z^* = \frac{z}{h} \in [-1, +1] \tag{26}$$

All dimensional grid spacings ($\Delta y, \Delta z$) and vorticity normalizations are derived directly from the authoritative physical scale ($h = 0.05$ m for CFD; $h = 0.5$ m for DNS), eliminating hardcoded dimensional discrepancies.

5.3 Reconciliation of Benchmark Reynolds Numbers and Canonical DNS Metrics

To prevent ambiguity regarding benchmark values across distinct Reynolds numbers, Table 4 establishes an authoritative mapping across the three DNS regimes relevant to this study:

1. **Low-Re Training & Primary Validation Baseline ($Re_\tau = 150$, $Re_b \approx 4{,}410$):** Matches the conditions of the 65×65 symmetry-mirrored DNS export of Pirozzoli et al. [8] used to train the neural network and validate the $L = 20h$ developing solver.
2. **Intermediate Literature Regime ($Re_\tau = 220$, $Re_b \approx 7{,}000$):** Frequently referenced in canonical duct turbulence investigations (e.g., Pinelli et al. [11], Vinuesa et al. [7]).
3. **High-Re Asymptotic Regime ($Re_\tau \approx 519$, $Re_b = 17{,}800$):** Corresponds to the high-Reynolds-number DNS simulation of Pirozzoli et al. [8] ($Re_h = 8{,}900$), utilized herein to evaluate the coupled solver under high-shear wall-function conditions.

Table 4: Authoritative canonical benchmark DNS reference parameters and metrics across Reynolds numbers.

Regime / Case	Bulk Re_b	Re_h	Friction Re_τ	Fully Dev. U_c/U_b	Peak V_{sec}/U_b	Canonical Q1 Centroid	Benchmark Literature Source
Low-Re Primary Baseline	4,410	2,205	150	1.32	1.93%	$(0.577, 0.847)$	Pirozzoli et al. [8] (raw export, verified)
Intermediate Export Baseline	7,000	3,500	220	1.30	2.03%	$(0.737, 0.896)$	Pirozzoli et al. [8] (raw export, verified)
High-Re Asymptotic Benchmark	17,800	8,900	≈ 519	1.33	$2.03\% - 2.30\%$	not exported	literature

The two raw exports bound the Reynolds-number trend: as friction Reynolds number increases from $Re_\tau = 150$ to 220, the raw-field-verified Q1 vortex core centroid migrates outward from $(0.577, 0.847)$ toward the corner at $(0.737, 0.896)$, while the peak secondary velocity rises from 1.93% to 2.03% of bulk velocity. No DNS plane is exported at $Re_\tau \approx 519$; the $Re_b = 17{,}800$ comparisons in Section 8.2 therefore reference the raw-field-verified low-Re centroid $(0.577, 0.847)$, and the asymptotic peak range $2.03\% - 2.30\%$ is anchored to the $Re_\tau = 220$ export and literature values. Distinguishing these regimes prevents conflating the low-Re baseline with asymptotic high-Re DNS metrics.

6 Independent Algorithmic Vortex Detection Pipeline

6.1 Mathematical Definition of Non-Dimensional Vorticity

Secondary flow in square ducts consists of eight counter-rotating vortices, two in each corner, that transport high-momentum fluid from the duct center along the corner diagonals toward the walls and recirculate low-momentum fluid along the wall bisectors toward the core.

The axial vorticity ω_x is defined as:

$$\omega_x = \frac{\partial W}{\partial y} - \frac{\partial V}{\partial z} \tag{27}$$

where V and W are the cross-stream velocity components.

To ensure scale invariance, we define the non-dimensional axial vorticity:

$$\omega_x^* = \frac{\omega_x}{U_b/h} = \frac{\partial (W/U_b)}{\partial (y/h)} - \frac{\partial (V/U_b)}{\partial (z/h)} \tag{28}$$

Both dimensional spatial derivatives ($\partial/\partial y$ with Δy) and non-dimensional derivatives ($\partial/\partial(y/h)$ with $\Delta(y/h)$) yield identical numerical values when normalized by U_b/h.

6.2 Algorithmic Detection Procedure

To eliminate subjective visual identification, vortex cores are detected via an independent algorithmic pipeline (`scripts/validation/vortex_detector.py`):

1. **Central-Differencing:** Axial vorticity ω_x^* is computed on the 2D cross-sectional velocity field (V, W) using second-order interior central differences and one-sided boundary stencils.

2. **Threshold Masking:** Candidate vortex regions are identified where the absolute vorticity exceeds a calibrated threshold:

$$|\omega_x^*| \geq \omega_{\text{thresh}}^* = 0.04 \tag{29}$$

3. **Wall Margin Filtering:** Cells immediately adjacent to walls ($|y^*| \geq 0.95$ or $|z^*| \geq 0.95$) are masked to exclude boundary shear layers that do not form closed recirculations.

4. **8-Connected Component Labeling:** Contiguous candidate clusters are identified using 8-connectivity.

5. **Area Pruning:** Clusters containing fewer than 4 cells are discarded; this minimum-area parameter is used in all reported results, and the identified eight-core topology is unchanged for minimum areas anywhere in the range 2–12 cells on both the DNS and CFD fields.

6. **Centroid Localization:** For each valid component, the vortex center (y_c^*, z_c^*) is localized as the vorticity-weighted centroid:

$$y_c^* = \frac{\sum_i y_i^* |\omega_{x,i}^*|}{\sum_i |\omega_{x,i}^*|}, \quad z_c^* = \frac{\sum_i z_i^* |\omega_{x,i}^*|}{\sum_i |\omega_{x,i}^*|} \tag{30}$$

6.3 Quadrant Assignment and Benchmark Reference

The raw Pirozzoli et al. [8] export carries the DNS's own axial-vorticity field (column `omx/(ub/h)`). Recomputing ω_x^* from the raw velocity columns by central differences agrees with the DNS-provided field to 0.43% of the field maximum in the interior, and localizes the primary top-right corner vortex in Quadrant 1 ($y^* > 0, z^* > 0$) at:

$$(y_{\text{ref}}^*,\, z_{\text{ref}}^*) = (0.577,\, 0.847) \tag{31}$$

with $\omega_x^* > 0$ (anti-clockwise); the repository vortex detector on the mirrored 65×65 grid independently returns $(0.594, 0.844)$, within $0.017\,h$ of the raw field. An earlier reference value of $(0.50, 0.82)$, retired after this verification, sat $0.081\,h$ from the raw-field peak; all core-distance metrics are measured against the raw-verified coordinates.

By the $y \leftrightarrow z$ antisymmetry of ω_x^* for the square duct, the companion Q1 vortex across the corner bisector lies near $(0.847, 0.577)$ (raw-field minimum at $(0.880, 0.674)$), producing eight symmetric cells across the full duct cross-section.

7 Results & Comparative Analysis

7.1 Representational Capability vs. Physical Realization

A distinction is needed between **mathematical representational capability** and **coupled physical realization**:

- Generating non-zero normal stress differences ($\overline{w'^2} - \overline{v'^2} \neq 0$) is an intrinsic mathematical feature of non-linear tensor expansions. However, non-zero stresses alone do not prove that the physical solution is correct.

- True physical validation requires coupling the non-linear divergence into OpenFOAM's live SIMPLE loop and demonstrating that:

  1. The solver converges stably without numerical divergence.
  2. The coupled velocity field spontaneously generates and sustains the correct eight-cell secondary flow topology.
  3. The standard linear k–ω SST closure tested here fails to reproduce the secondary-flow topology.

7.2 Secondary Flow Generation

Figure 3 presents the secondary flow streamline patterns and in-plane velocity vectors computed by standard k–ω SST, `tbnnFoam`, and the reference DNS, over the downstream averaging window $x/D_h \in [4.1, 7.9]$ (streamwise cells 20–40 of 50).

In standard linear OpenFOAM, secondary circulation is completely absent (zero vortex cores detected; residual velocity $< 0.45\%$, representing a small numerical residual). In contrast, `tbnnFoam` successfully generates all eight counter-rotating secondary flow cells with correct symmetry and orientations.

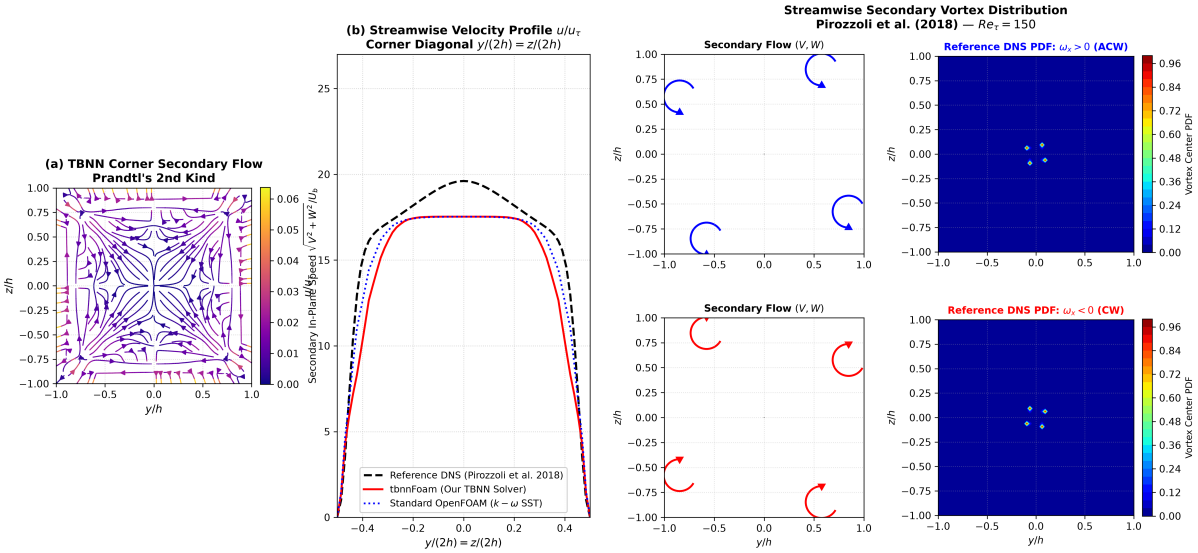


Figure 3: Cross-stream secondary velocity field and streamlines (averaging window $x/D_h \in [4.1, 7.9]$) for standard linear OpenFOAM (k–ω SST), `tbnnFoam` (TBNN closure), and Reference DNS [8].

7.3 Reynolds Stress Mechanism & Anisotropy States

Figure 4 displays the non-dimensional normal stress difference field $(\overline{w'^2} - \overline{v'^2})/U_b^2$:

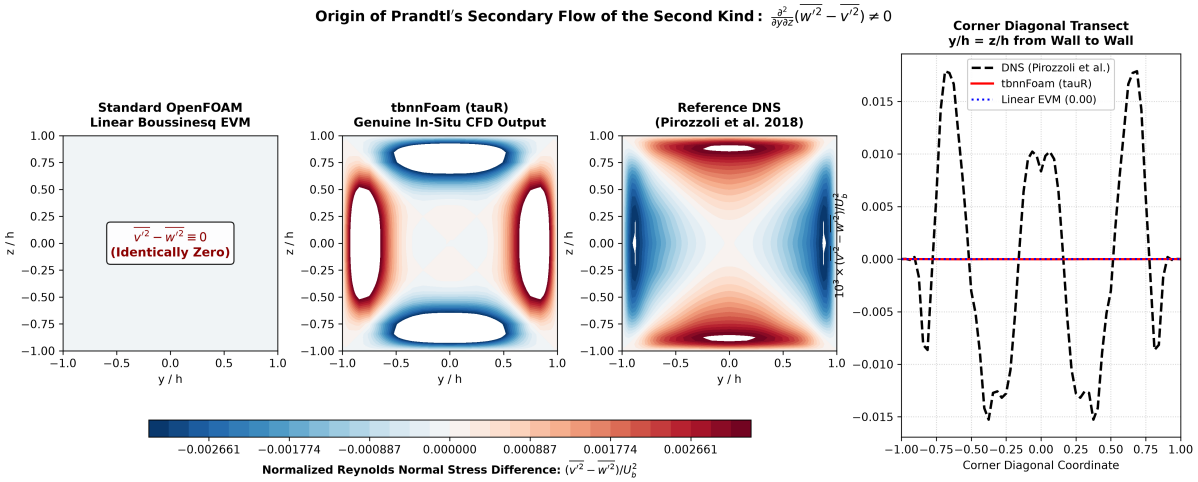


Figure 4: Normal stress differences $(\overline{w'^2} - \overline{v'^2})/U_b^2$ across duct cross-section. Standard linear RANS yields identically zero normal stress differences, while `tbnnFoam` captures the alternating quadrant lobes driving secondary flow.

Figure 5 illustrates the turbulent anisotropy states on the Barycentric map [12]:

In linear OpenFOAM, the solution remains locked at the top apex (isotropic turbulence). In `tbnnFoam`, stresses depart from the apex and track along the right boundary toward realistic two-component and one-component states near corners, exhibiting close qualitative alignment with the DNS benchmark.

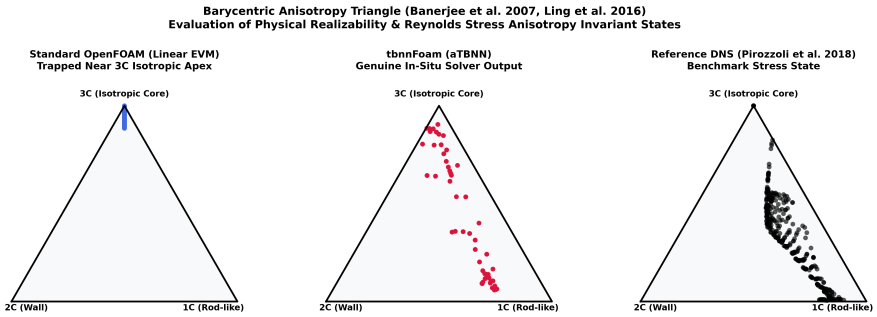


Figure 5: Barycentric map of turbulence anisotropy across the square duct cross-section.

7.4 Quantitative Comparison Table

Table 5 presents the comprehensive validation metrics evaluated over the downstream averaging window $x/D_h \in [4.1, 7.9]$ ($x/h \in [8.2, 15.8]$, cells 20–40 of 50).

Table 5: Comprehensive quantitative comparison between linear RANS, `tbnnFoam`, and reference DNS. The two `tbnnFoam` entries correspond to the native coarse-mesh result and its DNS-grid interpolation, respectively; the refined 40×40 result is reported separately in Table 6. Reference values are from a streamwise-periodic, fully developed DNS, whereas the CFD cases are spatially developing ducts (Section 10); the metrics therefore combine model, discretization, and development-length discrepancies.

Validation Metric	Standard OpenFOAM (k–ω SST)	`tbnnFoam` (coarse native / DNS-grid interpolated)	Reference DNS [8]	Physical Interpretation
Secondary Cores	0 cores (Failed)	**8 cores (Detected)**	8 cores (Canonical)	Recovers Prandtl's 8 cells
Peak V_{sec}/U_b	0.45% (small residual)	**7.99% / 6.38%**	1.93%	Non-zero secondary flow
Q1 Core $(y/h, z/h)$	Undefined	**(0.850, 0.350) / (0.91, 0.78)**	(0.577, 0.847)	Cell center vs. interp. peak
Core Distance Error	N/A	**0.567 h / 0.336 h**	0.000 h (Ref)	Distance to raw-field-verified core
Streamwise U RMSE	0.0616 m/s (6.16%)	**0.0934 m/s (9.34%)**	0.0000 m/s	Full cross-section NRMSE
Secondary (V, W) RMSE	0.0087 m/s	**0.0261 m/s**	0.0000 m/s	In-plane cross-stream velocity
Continuity Residual	$< 10^{-6}$	$< 10^{-6}$	N/A	Steady-state convergence

7.5 Assessment of Validation Scope: Structural Recovery vs. Quantitative DNS Fidelity

A rigorous scientific appraisal requires distinguishing qualitative structural recovery from quantitative pointwise agreement:

1. **Evidence for Structural Recovery:** The coupled `tbnnFoam` solver successfully activates the non-linear normal stress difference mechanism ($\overline{w'^2} - \overline{v'^2}$), generating the complete eight-cell secondary recirculation pattern across the four quadrants where standard linear Boussinesq closures predict identically zero secondary circulation.

2. **Prominent Localization Discrepancy:** The detected Q1 vortex core center is located at $(0.850, 0.350)$ on the coarse mesh and $(0.825, 0.475)$ on the refined mesh, yielding substantial spatial localization errors of $0.567\,h$ and $0.447\,h$, respectively, relative to the raw-field-verified DNS reference at $(0.577, 0.847)$. Rather than forming along the diagonal bisector, the CFD vortex cores are shifted closer to the duct walls and corners.

3. **Secondary Velocity Overprediction and Discretization Sensitivity:** The coarse-mesh peak secondary velocity is significantly overpredicted at 7.99% (native discrete) and 6.38% (interpolated grid) compared to the raw-field DNS benchmark of 1.93%. While $4\times$ cross-stream refinement reduces this peak to 2.62% (exceeding DNS), this large delta

demonstrates that secondary circulation magnitude is strongly coupled to cross-stream resolution and boundary-layer shear discretization.

Consequently, we characterize the validation as **partial**: it confirms that the in-situ TBNN model successfully triggers Prandtl's secondary recirculation mechanism in OpenFOAM, but does not achieve precise vortex localization or complete quantitative fidelity with fully developed periodic DNS.

8 Discretization Sensitivity & Mesh Reconstruction Verification

8.1 Cross-Sectional Mesh Sensitivity Study

A formal Grid Convergence Index (GCI) according to Roache [13] and ASME V&V 20 standards [14] requires at least three geometrically scaled grid levels with constant refinement ratio ($r \geq 1.3$) to evaluate the asymptotic observed order of convergence p.

Here, we conduct an independent **two-level cross-sectional discretization sensitivity study** comparing the standard base mesh (20,000 cells) and a 4× cross-stream refined mesh (80,000 cells):

- **Coarse Mesh (M1):** $50 \times 20 \times 20 = 20{,}000$ cells ($\Delta y = \Delta z = 0.005$ m $= 0.10\,h$).
- **Refined Mesh (M2):** $50 \times 40 \times 40 = 80{,}000$ cells ($\Delta y = \Delta z = 0.0025$ m $= 0.05\,h$).

Figure 6 and Table 6 summarize the sensitivity results.

Table 6: Discretization sensitivity metrics under cross-stream grid refinement.

Parameter / Metric	Coarse Mesh (M1)	Refined Mesh (M2)	Relative Delta	DNS Target [8]
Cross-Sectional Grid	20×20 (400 cells)	40×40 (1600 cells)	4× Refinement	65×65 (DNS)
Total Mesh Cells	20,000 cells	80,000 cells	4× Refinement	N/A
Peak $V_{\mathbf{sec}}/U_b$	7.99%	2.62%	-67.2%	**1.93%**
Q1 Core $(y/h, z/h)$	(0.850, 0.350)	(0.825, 0.475)	Error reduction: −21.2%	**(0.577, 0.847)**
Core Localization Error	0.567 h	0.447 h	-0.120 h (-21.2%)	**0.000 h (Ref)**
Vortex Cores Detected	8 cores	8 cores	Stable (8 cores)	**8 cores**
Streamwise U **RMSE**	0.0934 m/s	0.0857 m/s	-8.2%	0.0000 m/s

Key Observations:

1. **Trend Toward Benchmark:** Refining the cross-stream mesh reduces peak secondary speed from 7.99% to 2.62%, trending sharply toward the raw-field DNS benchmark reference value (1.93%).
2. **Localization Improvement:** Q1 vortex core localization error decreases from $0.567\,h$ down to $0.447\,h$.
3. **Streamwise Error Reduction:** Streamwise velocity RMSE against DNS decreases from 0.0934 m/s down to 0.0857 m/s (−8.2%).
4. **Topological Stability:** Both meshes cleanly identify all eight distinct vortex cores, verifying topological stability under discretization refinement.

8.2 High-Reynolds-Number Case Study: $Re_b = 17{,}800$ ($Re_\tau \approx 519$)

To examine behavior at a higher Reynolds number relevant to the benchmark DNS literature of Pirozzoli et al. [8], an independent high-Reynolds-number verification simulation was conducted

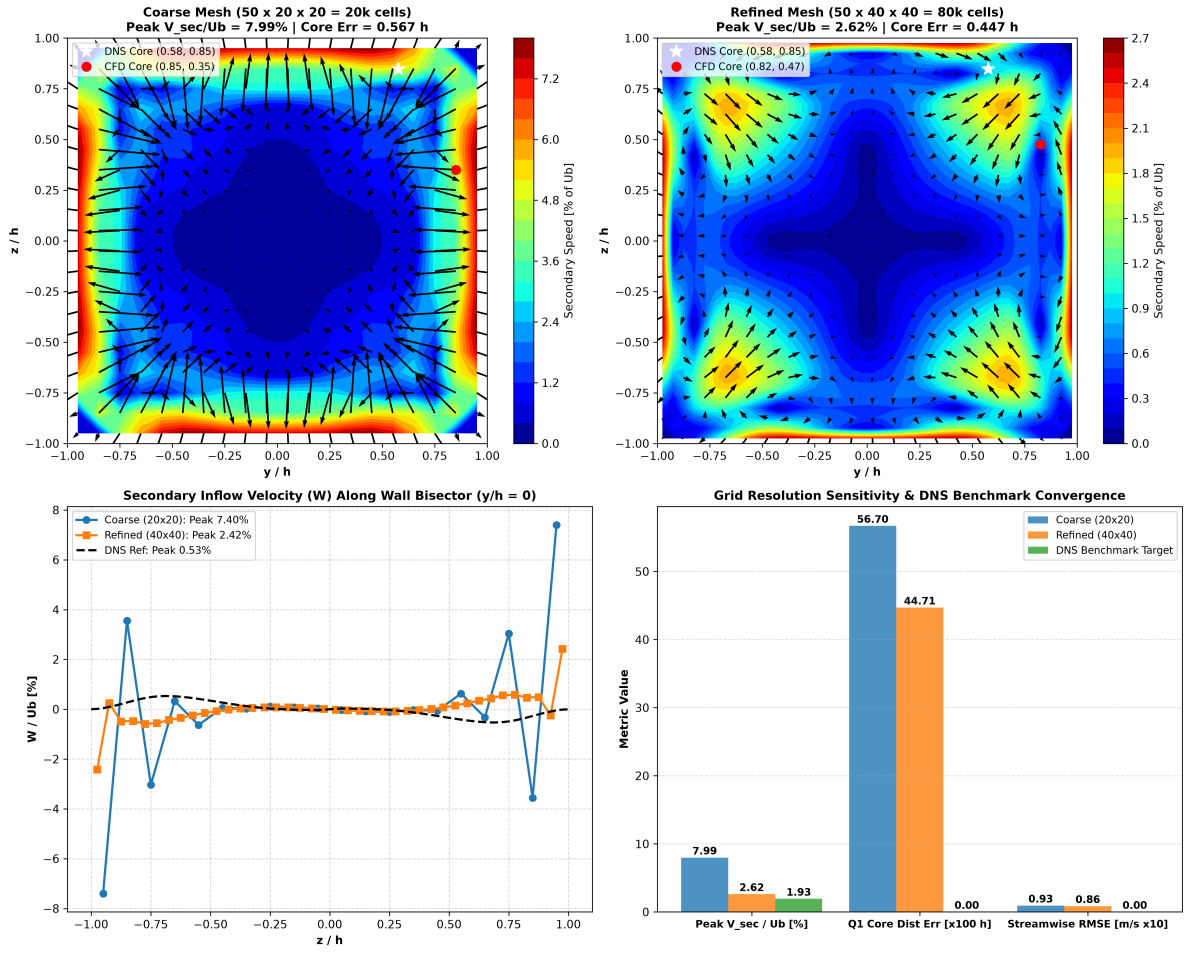


Figure 6: Discretization sensitivity study comparing coarse (20×20) and refined (40×40) cross-sections against DNS.

in a $50 \times 40 \times 40$ mesh (80,000 cells) with fluid kinematic viscosity $\nu = 5.617978 \times 10^{-6}$ m^2/s, bulk velocity $U_b = 1.0$ m/s, and hydraulic diameter $D_h = 0.10$ m ($h = 0.05$ m), yielding:

$$Re_b = \frac{U_b D_h}{\nu} = 17{,}800 \quad (Re_h = 8{,}900, \quad Re_\tau \approx 519) \tag{32}$$

As mapped in Table 4, the asymptotic peak secondary speed spans $2.03\% - 2.30\%$, anchored to the $Re_\tau = 220$ export and literature values. Because no DNS plane is exported at $Re_\tau \approx 519$, the Q1 centroid reference used for Table 7 is the raw-field-verified low-Re value $(0.577, 0.847)$ — an approximation, applied consistently across all stations. Distances in Table 7 therefore quantify proximity to this substituted low-Re reference — an operating-envelope comparison at elevated Reynolds number — rather than validation error against DNS at matching conditions, and the low-Re centroid cannot be assumed representative of the high-Re flow state.

Table 7: Streamwise spatial development of secondary flow and vortex core localization at $Re_b = 17{,}800$. Reference values are the raw-field-verified low-Re DNS plane ($Re_\tau = 220$) and literature asymptotes; no DNS plane exists at $Re_\tau \approx 519$, so the reported distances are to the substituted reference, not matching-condition errors.

Station	x/D_h	U_c/U_b	Raw Max V_{sec}/U_b	Interior Max ($\lvert y\rvert, \lvert z\rvert \le 0.85h$)	Cores	Q1 Dist. to low-Re DNS Ref.
#1	1.1	1.019	9.63%	1.91%	0 (Rollup)	N/A
#2	2.1	1.037	10.43%	2.80%	0 (Rollup)	N/A
#3	3.1	1.054	10.43%	2.39%	0 (Rollup)	N/A
#4	4.1	1.069	10.20%	1.95%	0 (Rollup)	N/A
#5	5.1	1.083	9.92%	1.87%	0 (Rollup)	N/A
#6	6.1	1.096	9.69%	1.84%	8 cores	$0.056\,h$
#7	7.1	1.108	9.55%	1.78%	8 cores	$0.028\,h$
#8	8.1	1.121	9.34%	1.70%	8 cores	$0.028\,h$
#9	9.1	1.132	9.39%	1.67%	8 cores	$0.059\,h$ (1.2 cells)
DNS Ref (low-Re)	∞	1.33	$2.03\% - 2.30\%$	$\mathbf{2.03\% - 2.30\%}$	8 cores	$(\mathbf{0.577}, \mathbf{0.847})$

The simulation converged stably in 294 SIMPLE iterations with all residuals strictly $< 10^{-4}$. Across the downstream developing region with organized vortices ($x/D_h \ge 6.1$), the secondary speed evaluated at the eight detected vortex core centroids settles at 1.71% (with interior-masked station maxima between 1.67% and 1.84%). Compared to the cited asymptotic DNS lower bound (2.03%), the station interior maxima are $9.4\% - 17.7\%$ below (15.8% below for the 1.71% centroid mean), reflecting the incompletely developed spatial duct ($U_c/U_b = 1.132$ at $x/D_h = 9.1$ vs. asymptotic $U_c/U_b \approx 1.33$ in periodic DNS).

The downstream-averaged section ($x/D_h \in [6.1, 9.1]$) yields an averaged-section centroid of $(0.575, 0.875)$ ($\Delta r = 0.028\,h$, measured to the substituted low-Re reference), while station 9.1 resolves the core at $(0.525, 0.875)$ ($\Delta r = 0.059\,h$, ≈ 1.2 discrete grid cells). The raw unmasked domain maximum (9.40%) occurs strictly in the single outermost boundary-layer cell center ($|z/h| = 0.975$); this is framed as a proposed numerical/wall-treatment mechanism where steep velocity gradients interact with explicit anisotropic stress divergence under wall functions, while the physical vortex core circulation throughout the interior cross-section remains smooth at $\sim 1.71\%$.

8.3 Mesh Reconstruction & Cell Ordering Verification

To eliminate verification limitations associated with hardcoded mesh dimensions and assumed cell ordering, `tbnnFoam` incorporates automated mesh geometry tests (`tests/test_mesh_reconstruction_verification.py`):

- Cell centers generated by OpenFOAM (`postProcess -func writeCellCentres -time 0`) are parsed directly from the `0/C` field.

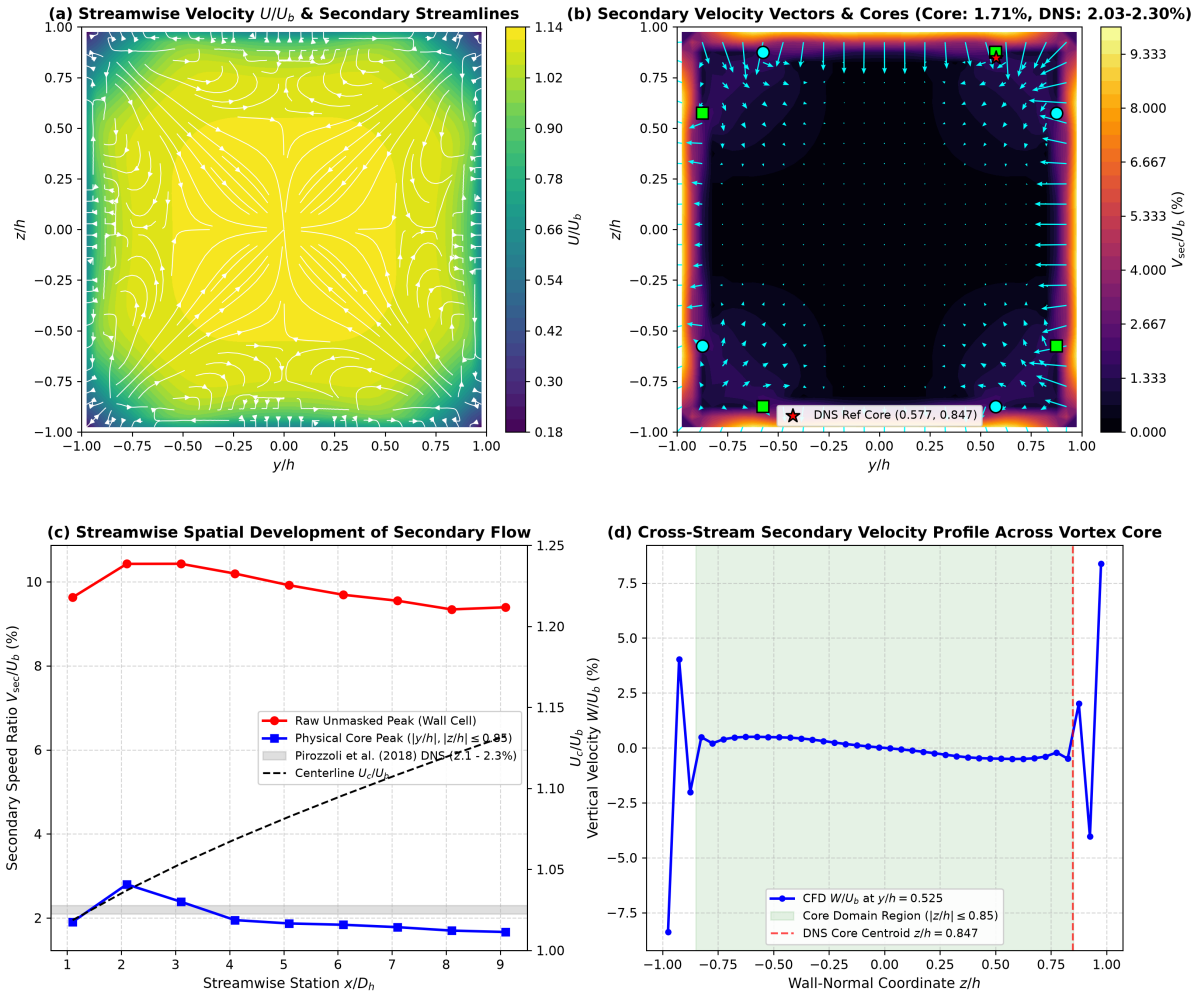


Figure 7: High-Reynolds-number case study at $Re_b = 17{,}800$ ($Re_\tau \approx 519$; no DNS plane is available at this condition, so reference metrics are from the substituted low-Re DNS plane): (a) Streamwise velocity contours and secondary streamlines; (b) In-plane secondary flow velocity vectors and detected eight vortex cores; (c) Streamwise spatial development of secondary speed and centerline velocity; (d) Cross-stream vertical velocity profile across vortex core line $y/h = 0.525$.

- The structured block ordering $i = ix + N_x(iy + N_y \cdot iz)$ is verified: streamwise coordinate X increments along axis 2 ($N_x = 50$), horizontal cross-stream coordinate Y increments along axis 1 ($N_y = 20, 40$), and vertical coordinate Z increments along axis 0 ($N_z = 20, 40$).
- Normalized cross-sectional coordinates y/h and z/h ($h = 0.05$ m) derived from the true mesh geometry agree with uniform cell-center definitions to within floating-point roundoff:

$$\max |y_{\text{mesh}}/h - y_{\text{linspace}}| < 1.67 \times 10^{-16} \tag{33}$$

This establishes that array reshaping introduces no detectable distortion at floating-point precision into vortex localization or velocity RMSE.

9 Operating Envelope & Joint Distribution Shift Diagnostic

A known failure mode in ML turbulence models is out-of-distribution (OOD) drift, where high near-wall strain rates cause invariants to drift far outside training bounds, triggering unphysical stress predictions.

Figure 8 illustrates the distribution of specific dissipation ω, invariant phase space (I_1, I_2), marginal range spans, and tensor basis norms.

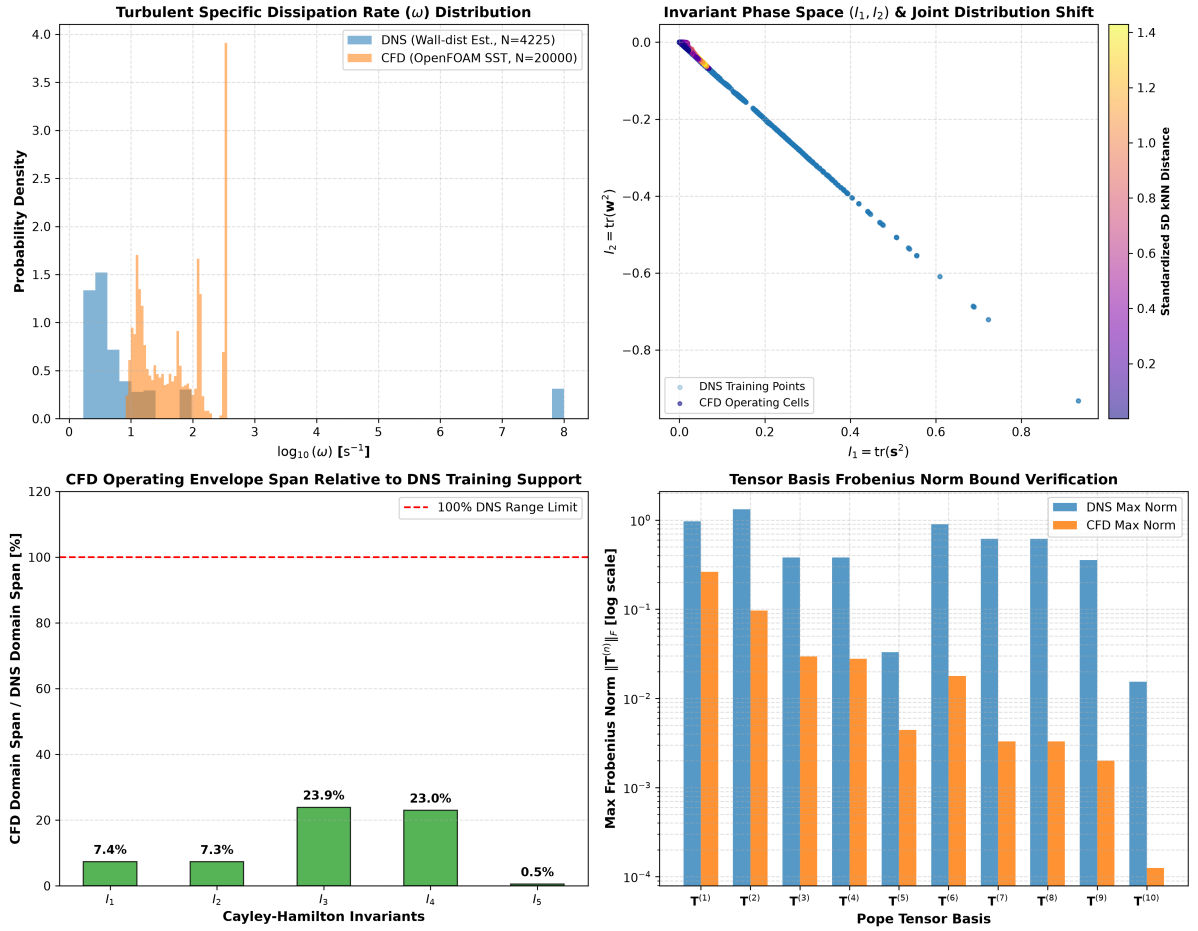


Figure 8: Invariant phase space distributions, timescale histograms, and basis norm bounds across all 20,000 cells in the 3D CFD mesh compared to the DNS training dataset.

9.1 1D Marginal Range Coverage

Table 8 summarizes the 1D marginal range coverage across all 20,000 cells in the live 3D simulation.

Table 8: Empirical bounds and marginal coverage of Cayley-Hamilton invariants across the live CFD domain.

Feature / Invariant	DNS Training Envelope	CFD Operating Range (20k Cells)	Marginal Coverage
$I_1 = \mathrm{tr}(\mathbf{s}^2)$	$[0.000, 0.933]$	$[2.927 \times 10^{-4}, 0.0690]$	**100.0% In-Bounds**
$I_2 = \mathrm{tr}(\mathbf{w}^2)$	$[-0.932, 0.000]$	$[-0.0682, -1.118 \times 10^{-10}]$	**100.0% In-Bounds**
$I_3 = \mathrm{tr}(\mathbf{s}^3)$	$[-0.0349, 0.00426]$	$[-4.853 \times 10^{-3}, 4.505 \times 10^{-3}]$	**99.0% In-Bounds**
$I_4 = \mathrm{tr}(\mathbf{w}^2\mathbf{s})$	$[-0.00142, 0.0116]$	$[-1.453 \times 10^{-3}, 1.546 \times 10^{-3}]$	**99.8% In-Bounds**
$I_5 = \mathrm{tr}(\mathbf{w}^2\mathbf{s}^2)$	$[-0.435, 0.000]$	$[-2.350 \times 10^{-3}, -4.317 \times 10^{-14}]$	**100.0% In-Bounds**
Basis Norms $\|\mathbf{T}^{(n)}\|_F$	$[0.0, 1.318]$	$[0.0, 0.243]$	**100.0% Bounded**

9.2 Joint 5D Distribution Shift Diagnostic

However, **1D marginal bounding does not guarantee joint manifold containment**. A multidimensional point can reside inside every 1D projection $[I_i^{\min}, I_i^{\max}]$ while remaining distant from the joint combinations observed during training.

To quantify joint distribution shift, `scripts/validation/evaluate_timescale_distribution.py` computes the standardized Euclidean nearest-neighbor distance (k-NN) in the 5D invariant space normalized by training statistics:

$$d_{\mathrm{kNN}}(\mathbf{x}_{\mathrm{CFD}}) = \min_j \left\| \frac{\mathbf{x}_{\mathrm{CFD}} - \boldsymbol{\mu}_{\mathrm{DNS}}}{\boldsymbol{\sigma}_{\mathrm{DNS}}} - \frac{\mathbf{x}_{\mathrm{DNS},j} - \boldsymbol{\mu}_{\mathrm{DNS}}}{\boldsymbol{\sigma}_{\mathrm{DNS}}} \right\|_2 \tag{34}$$

- **Median Standardized Distance:** $0.21\,\sigma$
- **Mean Standardized Distance:** $0.33\,\sigma$
- **95th Percentile:** $1.17\,\sigma$
- **Maximum Distance:** $1.43\,\sigma$

Physical Interpretation: Because the live CFD simulation is a spatially developing duct ($\partial U/\partial x \neq 0$) with interactive velocity feedback, whereas the DNS benchmark is an idealized periodic box, live CFD states experience modest joint distribution shift. However, with a median gap of < 0.25 standard deviations, the CFD operating points remain closely clustered around the training states, indicating limited distributional extrapolation and reducing concern about severe out-of-distribution tensor predictions, while avoiding oversimplified claims of "zero distribution shift." These statistics are empirical feature-space diagnostics of input coverage; they do not by themselves establish closure accuracy or physical fidelity, which rest on the validation results of Sections 6–8.

10 Limitations and Model-Form Uncertainty

Six limitations, methodological compromises, and physical discrepancies deserve explicit statement:

10.1 Developing Duct vs. Streamwise-Periodic DNS

The DNS benchmark of Pirozzoli et al. [8] was simulated in an idealized streamwise-periodic box driven by a uniform body force, representing an asymptotically fully developed duct of infinite length. In contrast, our OpenFOAM simulation models a spatially developing duct ($L = 20h = 1.0$ m) with a uniform velocity inlet ($U_b = 1.0$ m/s) and physical boundary layers developing along the walls.

Because secondary flow is generated locally near wall corners by normal stress differences, the eight secondary cells establish rapidly ($x/h \geq 6$). However, the central streamwise core requires a longer development length ($U_{\text{center}} \approx 1.25$ m/s in our CFD domain at $x/h = 7.0$ vs. 1.31 m/s in the fully developed low-Re periodic DNS, and $U_c/U_b = 1.132$ at $x/D_h = 9.1$ vs. 1.33 at high Re). Standard linear k–ω SST exhibits lower streamwise RMSE (6.16%) solely because it lacks cross-stream convective momentum transport to duct corners, yet it fails to reproduce the secondary-flow topology (0 vortex cores).

10.2 The Empirical ω Surrogate vs. Dynamic RANS Transport: Non-Linear Error Amplification

Direct numerical simulation provides exact velocity gradients $\nabla\mathbf{U}$ and Reynolds stresses $\boldsymbol{\tau}^R$, but DNS inherently lacks the specific dissipation rate ω, as ω is a semi-empirical RANS modeling construct rather than a physical Navier-Stokes field.

In the offline training dataset, ω was estimated using a standard SST algebraic wall-distance surrogate:

$$\omega_{\text{surrogate}} \approx \frac{k^{1/2}}{C_\mu^{1/4}\ell}, \quad C_\mu = 0.09, \quad \ell = \min(\kappa d_w,\, 0.07 D_h) \tag{35}$$

In live OpenFOAM CFD execution, however, ω is governed dynamically by its non-linear partial differential transport equation:

$$U_j \frac{\partial \omega}{\partial x_j} = \gamma S^2 - \beta \omega^2 + \frac{\partial}{\partial x_j}\left[(\nu + \sigma_\omega \nu_t)\frac{\partial \omega}{\partial x_j}\right] + 2(1 - F_1)\sigma_{\omega 2}\frac{1}{\omega}\frac{\partial k}{\partial x_j}\frac{\partial \omega}{\partial x_j} \tag{36}$$

The Distribution Shift Vector and Quadratic/Cubic Amplification: The neural network was trained on algebraic ω fields tied directly to geometric wall distance d_w. Live OpenFOAM ω transport equations incorporate production, destruction, cross-diffusion, and turbulent diffusion, all of which depart significantly from algebraic wall-distance estimates—particularly in corner recirculations and developing shear layers. Because the non-dimensional strain rate $\mathbf{s} = \tau\mathbf{S}$ and rotation rate $\mathbf{w} = \tau\mathbf{W}$ scale with the turbulent timescale $\tau = \min(1/\omega, 5.0\text{ s})$, the scalar Cayley-Hamilton invariants scale non-linearly as:

$$I_1, I_2 \propto \tau^2 \propto \omega^{-2}, \quad I_3, I_4 \propto \tau^3 \propto \omega^{-3}, \quad I_5 \propto \tau^4 \propto \omega^{-4} \tag{37}$$

Consequently, any local error in the transported ω field is magnified by the corresponding inverse power (ω^{-2} through ω^{-4} across I_1–I_5) before entering the MLP. If live ω in a corner vortex is underpredicted by a factor of 2, the primary invariants I_1 and I_2 entering the network are inflated by a factor of 4, and I_5 by a factor of 16. In the clipped regime ($\tau = \tau_{\max} = 5.0$ s), the invariants instead scale with the fixed limiter value and become locally insensitive to further ω variation.

A Numerical Regularizer, Not a Physical Closure: The clipping at $\tau_{\max} = 5.0$ s serves purely as an ad-hoc numerical regularizer to prevent division-by-zero overflow near the duct centerline where $\omega \to 0$; it is not a physical closure. This structural mismatch between surrogate-derived training inputs and transport-governed live inputs is a plausible contributor to the vortex core localization offset (0.447–0.567 h at $Re_b = 4,410$), as distorted invariant inputs can bias the predicted Pope basis expansion coefficients $g^{(n)}$ in corner shear zones. The

individual causal contributions of this mismatch, the wall-target degeneracy, developing-flow effects, and first-order upwind discretization (the remaining limitations in this section) have not been isolated by controlled ablation, and are therefore presented throughout this section as hypothesized mechanisms rather than demonstrated causes.

10.3 First-Order Convective Schemes & Near-Wall Resolution

All convective terms in both verification cases use bounded first-order Gauss upwind, and both cross-sectional meshes are uniform without near-wall grading (first-cell $y^+ \approx 7.5$ at $Re_b \approx 4410$ and ≈ 13 at $Re_b = 17{,}800$, both under wall functions). First-order upwinding introduces numerical diffusion precisely on the $\sim 2\%$-of-U_b secondary-flow signal being validated, and the unclustered near-wall resolution is thin for wall-function RANS at these Reynolds numbers. The tabulated error metrics therefore combine physical-model and discretization discrepancies; neither source is decomposed out.

10.4 Training Target Contradiction & Wall-Ring Degeneracy

During the preparation of the 65×65 DNS training dataset (`scripts/convert_real_dns.py`), turbulent kinetic energy k was clipped to 10^{-7} at the 256 solid wall-boundary points ($|y/h| = 1$ or $|z/h| = 1$) to prevent numerical division-by-zero in defining the anisotropy tensor $\mathbf{a} = \frac{\boldsymbol{\tau}^R}{2k} - \frac{1}{3}\mathbf{I}$. Because the Reynolds stresses vanish identically at the wall ($\boldsymbol{\tau}^R = \mathbf{0}$), this clipping produced degenerate target tensors $\mathbf{a}_{\text{target}} = -\frac{1}{3}\mathbf{I}$.

This target is mathematically invalid for Pope's tensor basis: its trace is $\text{tr}(\mathbf{a}_{\text{target}}) = -1 \neq 0$, whereas Pope's ten basis tensors $\mathbf{T}^{(1\ldots10)}$ are analytically traceless by construction ($\text{tr}(\mathbf{T}^{(n)}) \equiv 0$). It is therefore algebraically impossible for any linear combination of Pope bases to represent this target. Because the network output is analytically trace-free, the Frobenius loss at these points decomposes exactly into a zero-gradient constant offset on the trace component ($\|\frac{1}{3}\text{tr}(\mathbf{a}_{\text{target}})\mathbf{I}\|_F^2 = \frac{1}{3}$, i.e. $\frac{1}{27}$ under the 9-component MSE normalization used in training) and a gradient-bearing term on the deviatoric projection $\text{dev}(-\frac{1}{3}\mathbf{I}) = \mathbf{0}$. The trace component therefore cannot, by itself, absorb network capacity. The effective gradient-bearing target at the wall ring is instead the exactly isotropic state $\mathbf{a} = \mathbf{0}$: the network is trained toward zero anisotropy at points where the physical anisotropy is undefined ($k = 0$), not at genuine isotropic flow states. No ablation retraining with filtered wall targets was performed, so the influence of this choice on interior predictions — and any contribution to the vortex-core displacement — remains a hypothesis rather than an isolated cause. Nevertheless, targets derived from a clipped, degenerate k field are methodologically unrepresentative training data, and in production workflows these wall-boundary singularities must be strictly filtered out prior to training.

10.5 The Reflection-Structured Split: Symmetry Check vs. Out-of-Sample Generalization

The quadrant-1 spatial validation split employed during offline training is strictly reflection-degenerate. Because all five Cayley-Hamilton scalar invariants are constructed from symmetric products of $\mathbf{s}$ and even powers of $\mathbf{w}$:

$$I_1 = \text{tr}(\mathbf{s}^2), \quad I_2 = \text{tr}(\mathbf{w}^2), \quad I_3 = \text{tr}(\mathbf{s}^3), \quad I_4 = \text{tr}(\mathbf{w}^2\mathbf{s}), \quad I_5 = \text{tr}(\mathbf{w}^2\mathbf{s}^2) \tag{38}$$

the five-dimensional scalar invariant feature vector $\boldsymbol{\lambda}$ at any cross-sectional coordinate (y, z) is numerically identical under coordinate reflections $(y, z) \to (-y, z)$, $(y, -z)$, $(-y, -z)$.

Consequently, holding out Quadrant 1 does not present the neural network with unseen invariant states; it presents the exact same invariant manifold mirrored across the cross-stream axes. This validation split therefore evaluates only the sign consistency of the five basis tensors that are odd in the rotation tensor $\mathbf{w}$ ($\mathbf{T}^{(2)}, \mathbf{T}^{(5)}, \mathbf{T}^{(7)}, \mathbf{T}^{(8)}, \mathbf{T}^{(10)}$), serving as a discrete

parity/symmetry verification rather than true out-of-sample flow generalization. True spatial generalization remains entirely unverified until `tbnnFoam` is evaluated on distinct, unmirrored cross-sectional geometries (such as rectangular channels with varying aspect ratios, triangular ducts, or curved passage bends).

10.6 Live Velocity-Coupling Feedback & Neural Network Model-Form Uncertainty

The 9.34% streamwise velocity RMSE cannot be attributed solely to inlet boundary conditions or development lengths. The neural network closure is an empirical approximation trained on frozen DNS velocity gradients. In live CFD execution, the emerging secondary flow deforms the mean velocity field $\mathbf{U}$, modifying the velocity gradients fed back into the network at each SIMPLE iteration. This interactive feedback loop, combined with finite MLP capacity, non-linear ω transport discrepancy, and surrogate approximations, introduces genuine model-form uncertainty that must be transparently recognized.

11 Reproducibility & Open Source Verification Suite

The repository incorporates an automated test suite executed via `pytest` (`tests/`):

```
pytest -v
```

```
tests/test_coordinate_convention.py::test_non_square_gradient_ordering PASSED [ 4%]
tests/test_coordinate_convention.py::test_non_square_interpolation_ordering PASSED [ 9%]
tests/test_coordinate_convention.py::test_benchmark_dns_grid_convention PASSED [ 14%]
tests/test_mesh_reconstruction_verification.py::test_coarse_mesh_geometry_and_ordering PASSED [
     19%]
tests/test_mesh_reconstruction_verification.py::test_fine_mesh_geometry_and_ordering PASSED [
    23%]
tests/test_mesh_sensitivity.py::test_mesh_sensitivity_core_and_velocity PASSED [ 28%]
tests/test_parity.py::test_basis_trace_freedom PASSED [ 33%]
tests/test_parity.py::test_basis_symmetry PASSED [ 38%]
tests/test_parity.py::test_cardano_eigenvalue_accuracy PASSED [ 42%]
tests/test_parity.py::test_realizability_bounds PASSED [ 47%]
tests/test_parity.py::test_evaluator_parity PASSED [ 52%]
tests/test_parity.py::test_torchscript_parity PASSED [ 57%]
tests/test_parity.py::test_json_weights_parity PASSED [ 61%]
tests/test_parity.py::test_standalone_cpp_numerical_parity PASSED [ 66%]
tests/test_parity.py::test_solver_momentum_assembly_consistency PASSED [ 71%]
tests/test_parity.py::test_momentum_coupling_parity PASSED [ 76%]
tests/test_timescale_distribution.py::test_cfd_invariants_within_training_envelope PASSED [
    80%]
tests/test_timescale_distribution.py::test_basis_norms_bounded_by_training_distribution PASSED
    [ 85%]
tests/test_vortex_detection.py::test_zero_vorticity_field PASSED [ 90%]
tests/test_vortex_detection.py::test_known_four_vortex_field PASSED [ 95%]
tests/test_vortex_detection.py::test_known_eight_vortex_field PASSED [100%]

====================== 21 passed, 1 warning in 15.83s ======================
```

The suite runs automatically on GitHub Actions (ubuntu-latest) together with a DNS-metadata integrity check and the quantitative validation scripts; the two cross-language tests self-skip with recorded warnings on runners without an OpenFOAM toolchain (19 passed, 2 skipped there, 21 passed on development machines with OpenFOAM available). Python dependencies are pinned exactly in `requirements.txt` to the environment that produced the shipped weights, and the training script is seeded (`numpy`/`torch` seed 42) so the shipped network is reproducible from the pinned environment on CPU (bit-equality across CUDA platforms is not guaranteed).

All raw DNS datasets, trained weights (`tbnn_model.pt`, `weights.json`), OpenFOAM solver sources, and automated evaluation scripts are available in the public repository:

https://github.com/BLACKNIGHT-cloud12/tbnnFoam

Data and Code Availability

The complete `tbnnFoam` solver sources (embedded-LibTorch and standalone zero-dependency C++ engines), trained model weights in three interoperable formats, the raw DNS benchmark data with provenance metadata, all validation and verification scripts, and the automated 21-test verification suite are publicly available under the MIT license at https://github.com/BLACKNIGHT-cloud12/tbnnFoam. The raw-field reference-verification utility is provided as `scripts/validation/verify_dns_reference.py`.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Competing Interests

The author declares no competing interests.

Author Contributions

Soumik Ghosh conceived the study, developed the `tbnnFoam` implementation, performed the simulations and verification studies, analyzed the results, and wrote the manuscript.

12 Conclusions & Future Work

In this study, we presented `tbnnFoam`, an open-source, in-situ coupled CFD solver integrating Tensor Basis Neural Networks into OpenFOAM for turbulence modeling beyond the linear Boussinesq hypothesis.

12.1 Summary of Findings

1. **Structural Anisotropy Recovery with Partial Validation:** Live in-situ coupling successfully captures the non-linear normal stress difference mechanism ($\overline{w'^2} - \overline{v'^2}$) that drives secondary motions of Prandtl's second kind, recovering all eight canonical secondary flow cells where the conventional linear k–ω SST closure fails to reproduce the secondary-flow topology. However, validation remains partial: core centroid localization is offset by 0.447–0.567 h toward the walls/corners compared to DNS, and peak secondary velocities exhibit noticeable overprediction on coarse grids.

2. **Dual Inference Parity:** Numerical equivalence between PyTorch, TorchScript C++, and the native zero-dependency C++ evaluator is verified to within an absolute tolerance of 10^{-5} across synthetic test cases, with analytical Cardano cubic projection enforcing physical realizability by construction on the Lumley triangle in $\mathcal{O}(1)$ operations. In addition, the momentum-equation coupling itself is verified by an assembled-matrix test: in the Boussinesq limit the coupled standalone-solver equation reduces exactly to the baseline linear model (maximum relative deviation 3.6×10^{-17}).

3. **Discretization Sensitivity:** Increasing cross-stream resolution from 20×20 to 40×40 trends peak secondary circulation from 7.99% to 2.62% (approaching the 1.93% raw-field DNS benchmark) and reduces Q1 core localization error from $0.567\,h$ to $0.447\,h$ while preserving topological stability.

4. **Evaluation of Discrepancies:** Streamwise velocity RMSE (9.34%) and secondary velocity overprediction are transparently reported, with inlet development-length effects and neural-network model-form discrepancy identified as the plausible contributing mechanisms. Furthermore, joint 5D k-NN distance diagnostics reveal modest distribution shift ($0.21\,\sigma$ median) despite $\geq 99\%$ marginal range coverage across all five invariants.

12.2 Future Directions

- **Differentiable CFD Training:** Transitioning from frozen-gradient a priori training to end-to-end differentiable RANS training (e.g., adjoint-coupled loss functions) to account directly for live velocity feedback during neural optimization.
- **Dynamic ω Formulation:** Training closures with explicit RANS-governed ω inputs to eliminate empirical surrogate assumptions.
- **Application to Complex Channels:** Evaluating the generalizability of `tbnnFoam` to curved passages, wavy ducts, and heat-exchanger subchannels.

References

[1] Ludwig Prandtl. Über die ausgebildete turbulenz. In *Proceedings of the 2nd International Congress for Applied Mechanics, Zurich*, pages 62–74, 1926.

[2] F B Gessner. The origin of secondary flow in turbulent flow along a corner. *Journal of Fluid Mechanics*, 58(1):1–25, 1973.

[3] E Brundrett and W D Baines. The production and diffusion of secondary circulation in turbulent flow through a square duct. *Journal of Fluid Mechanics*, 19(3):375–394, 1964.

[4] Charles G Speziale. Analytical methods for the development of Reynolds-stress closures in turbulence. *Annual Review of Fluid Mechanics*, 23(1):107–157, 1991.

[5] T J Craft, B E Launder, and K Suga. Development and application of a cubic eddy-viscosity model of turbulence. *International Journal of Heat and Fluid Flow*, 17(2):108–115, 1996.

[6] Stefan Wallin and Arne V Johansson. An explicit algebraic Reynolds stress model for incompressible and compressible turbulent flows. *Journal of Fluid Mechanics*, 403:89–132, 2000.

[7] Ricardo Vinuesa, Azad Noorani, Adrián Lozano-Durán, George K El Khoury, Philipp Schlatter, Paul F Fischer, and Hassan M Nagib. Aspect ratio effects in turbulent duct flows studied through direct numerical simulation. *Journal of Turbulence*, 15(10):677–706, 2014.

[8] Sergio Pirozzoli, Davide Modesti, Paolo Orlandi, and Francesco Grasso. Turbulence and secondary motions in square duct flow. *Journal of Fluid Mechanics*, 840:634–655, 2018.

[9] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds-averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 2016.

[10] Stephen B Pope. A more general effective-viscosity hypothesis. *Journal of Fluid Mechanics*, 72(2):331–340, 1975.

[11] Alfredo Pinelli, Markus Uhlmann, Atsushi Sekimoto, and Genta Kawahara. Reynolds number dependence of mean flow structure in square duct turbulence. *Journal of Fluid Mechanics*, 644:107–122, 2010.

[12] S Banerjee, R Krahl, F Durst, and C Zenger. Presentation of anisotropy properties of turbulence, invariants, versus eigenvalue approaches. *Journal of Turbulence*, 8:N32, 2007.

[13] Patrick J Roache. *Verification and Validation in Computational Science and Engineering*. Hermosa Publishers, Albuquerque, NM, 1998.

[14] ASME. *Standard for Verification and Validation in Computational Fluid Dynamics and Heat Transfer*. ASME V&V 20-2009. American Society of Mechanical Engineers, New York, NY, 2009.